

Computer Science & IT



Compiler Design



Parsers
Lecture No. 01



By- Ravindra Sir

Topics to be Covered



Topic

Parsers

Topic

Topic

Practice questions → Telegram channel

↓
all subjects

RBR Sir PW

Language
dependent
front end

- LA
- Syn ana
- Semantic ana
- ICG

Target-
dependent
backend

- CO
- TCG

Lexical analyzer:-

lexical analysis is the act of breaking down source text into a set of words called tokens.

Each token is found by matching sequential characters to patterns.

All tokens are defined with regular

grammar and lexical analyzer identifies strings of tokens and sends them to syntax analyzer for parsing.

Lexical Specification
(Regular expression)

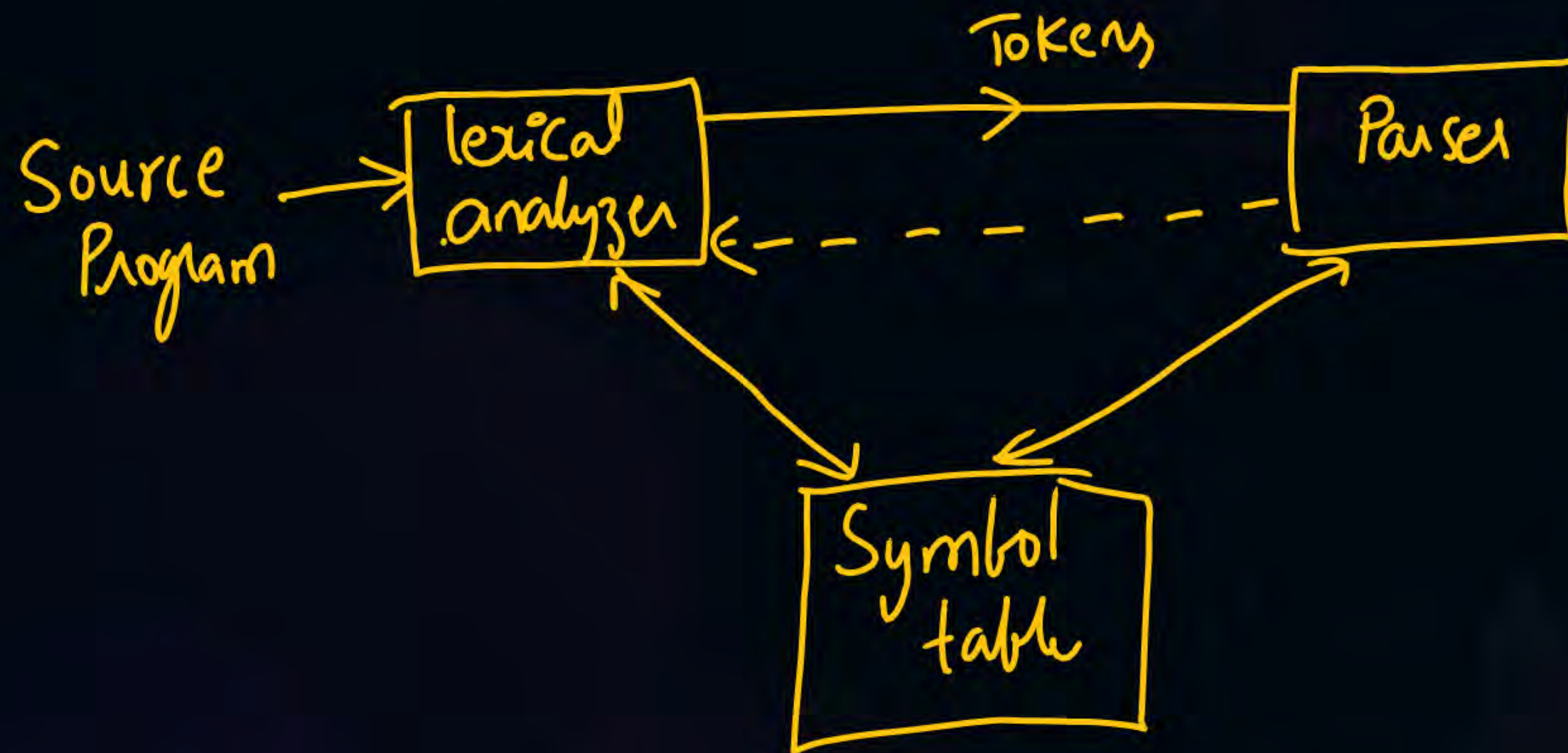
```

if (x >= 1) {
    x = x + 1
} else {
    x = x - 1;
}

```



IF, LP, id, Relop, Num, RP, LB,
 Id, ASGN, id, Addop, Num,
 RB, ELSE, LB
 id, ASGN, ID, Subop, Num, SC,
 RB



Secondary tasks of a lexical analyzer:-

1. Filtering comment lines and white space characters

White space characters are tab, space, newline

Comments are not required during translation of code

2. Associating the end messages with line numbers.
A lexical analyzer reads the source code character by character, while reading it remembers the line number

Tokens, patterns, lexemes



Token is a group of characters with logical meaning

Ex: Id, Keyword, Num etc

Pattern: It is a rule that describes the characters that can be grouped into tokens

It is expressed as regular expression.

Ex:- Pattern/Rule for identifier is that it should start with a letter followed by any number of letters & digits

$[A-Z a-z][A-Z a-z 0-9]^*$ ✓

lexeme :- It is the actual text / character stream that matches with the pattern and recognised as Token



Gate

float key = 1.2 ;

Lexeme

float

key

=

1.2

;

Token

float

id

ASSgn

Num

SC

Pattern

float

A letter followed by any number of letters
or
digits

=
any numeric constant
;

Gate:- How many tokens are in following programs



```
|int| max(|x|, |y|)|  
    int |x|, |y|;
```

```
/* find max of x and y */ x
```

```
{ return (|x| > |y| ? |x| : |y|); }
```

```
}
```

a) 27 b) 26 ~~c) 25~~ d) 24

| printf | (| "%d Hai" | , | & | x |) | ; |

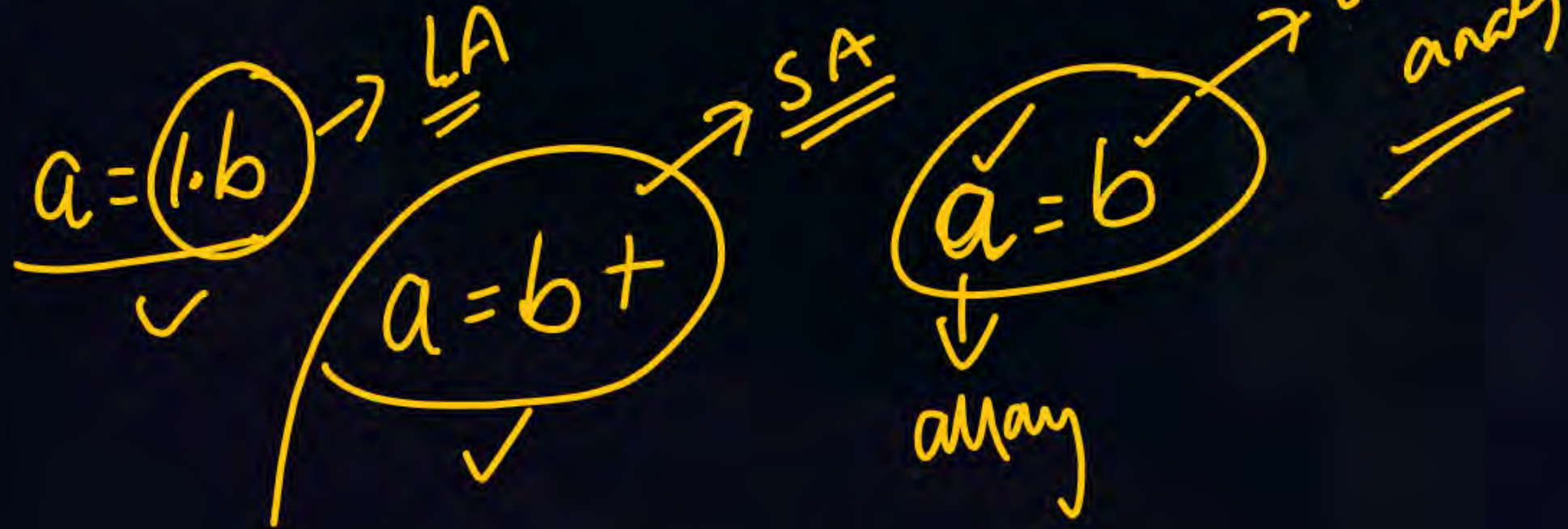
⑧

Errors detected in lexical analysis:-

- 1) Numeric literals are too long ✓
 - 2) long identifiers ✓
 - 3) Ill-formed numeric literals (1.a) ✓
 - 4) I/p character that are not in source language ✓✓
- programs called end handlers

int whiel;

whiel
{
}

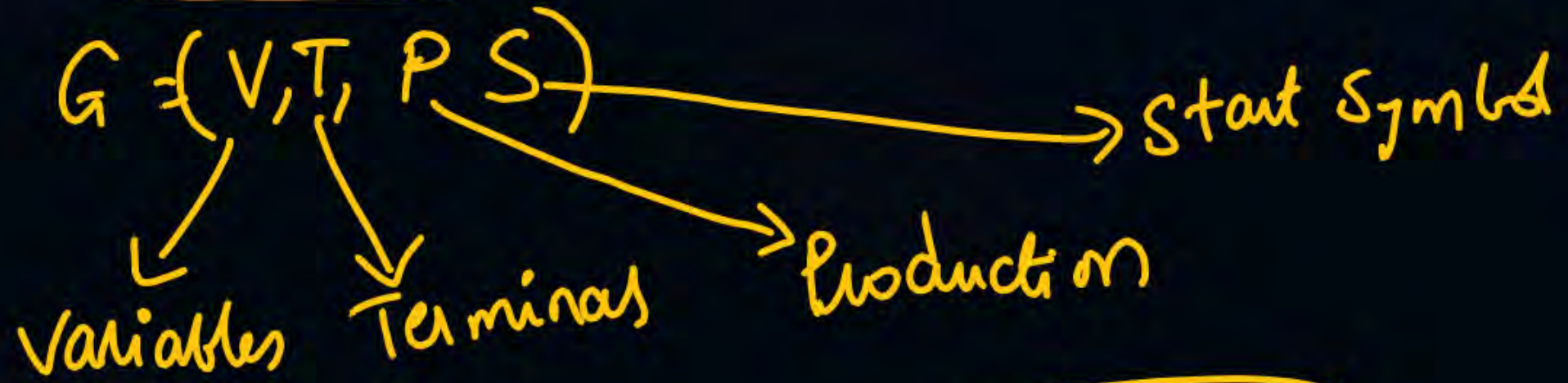


How to recover from errors:-

There are 4 approaches to recover from lexical errors:

- Delete: If there is a unknown character, then delete it
- Insert: Insert missing character (chr → char)
- Transpose: We can transpose two characters
 "whiel" can be corrected to "while"
- Replace: Replace a character by another character
 $chrr \Rightarrow char$

Grammars :-



$E \rightarrow E + E$
 $E \rightarrow E * E$
 $E \rightarrow id$

$V = \{E\}$
 $T = \{+, *, id\}$

$E \rightarrow E + E$
 $/ E * E$
 $/ id$

$E \rightarrow E + E$
 $\quad \quad \quad / E * E$
 $\quad \quad \quad / id$

$id + id * id$

LMD ✓

$E \Rightarrow \underline{E} + E$
 $\Rightarrow id + E$
 $\Rightarrow id + \underline{E} * E$
 $\Rightarrow id + id * E$
 $\Rightarrow id + id * id$

LMD ✓

$E \Rightarrow \underline{E} * E$
 $\Rightarrow \underline{E} + E * E$
 $\Rightarrow id + \underline{E} * E$
 $\Rightarrow id + id * E$
 $\Rightarrow id + id * id$

$E \rightarrow E + E$
 $\quad \quad \quad | E * E$
 $\quad \quad \quad | id$

$id + id * id$

LMD ✓

$E \Rightarrow \underline{E} + E$
 $\Rightarrow id + E$
 $\Rightarrow id + \underline{E} * E$
 $\Rightarrow id + id * E$
 $\Rightarrow id + id * id$

✓

LMD ✓

$E \Rightarrow \underline{E} * E$
 $\Rightarrow \underline{E} + E * E$
 $\Rightarrow id + \underline{E} * E$
 $\Rightarrow id + id * E$
 $\Rightarrow id + id * id$

✓

$$E \rightarrow E + E \\ / E * E \\ / id$$

$$id + id * id$$

RMD

$$E \Rightarrow E + \underline{E}$$

$$\Rightarrow E + E * \underline{E}$$

$$\Rightarrow E + E * id$$

$$\Rightarrow E + id * id$$

$$\Rightarrow id + id * id$$

✓

RMD

$$E \Rightarrow E * \underline{E}$$

$$\Rightarrow \underline{E} * id$$

$$\Rightarrow E + \underline{E} * id$$

$$\Rightarrow \underline{E} + id * id$$

$$\Rightarrow id + id * id$$

✓

$E \rightarrow E + E$
 $\quad \mid E * E$
 $\quad \mid id$

$id + id * id$

Parse tree :



✓ (CN)



✓ CISCO - ASRSK →

✓ ARP RFC

RFC

RA

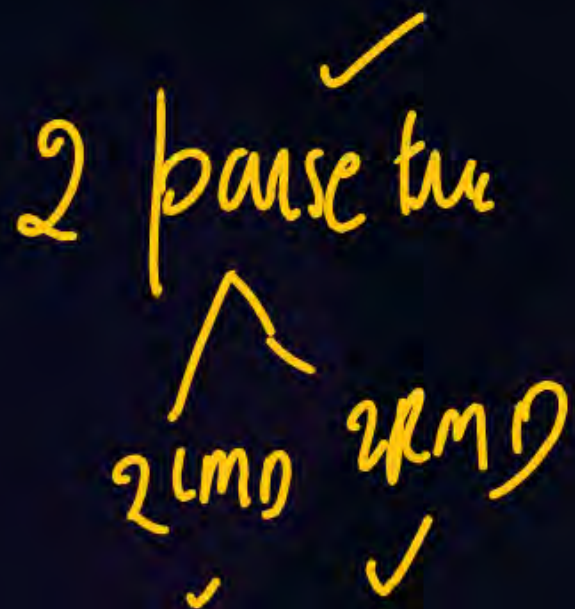
MTG

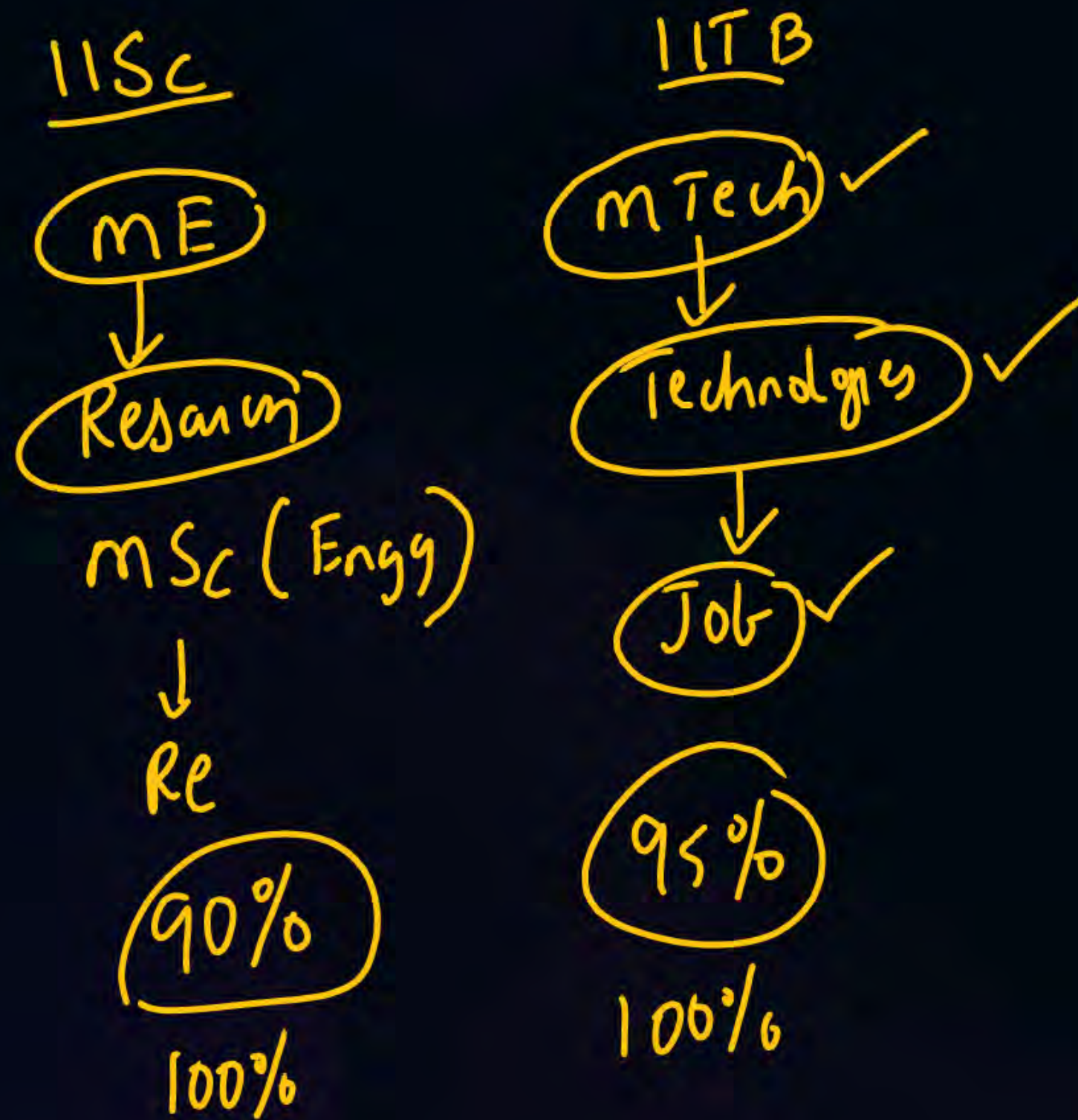
→ If for any grammar, for some string if we get
 more than 1 LMD & more than 1 RMD & more than one
 parse tree, the grammar is called ambiguous
grammar

→ No parser will accept ambiguous grammars except operator precedence parser.

→ Ambiguity of a grammar is undecidable

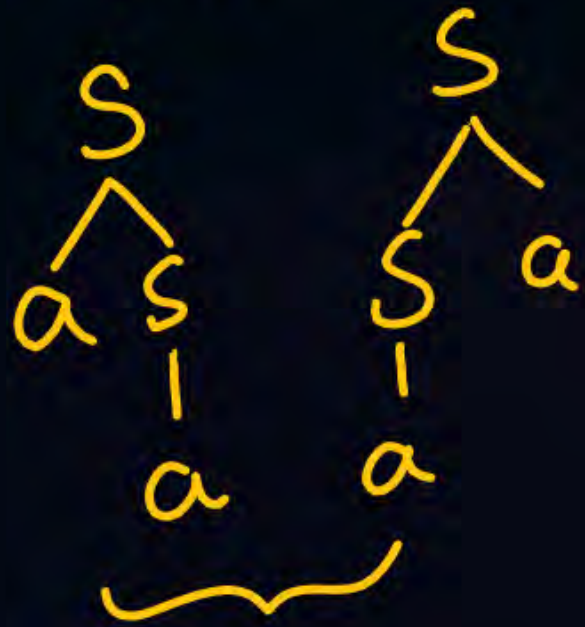
→ Removing ambiguity from a grammar is undecidable





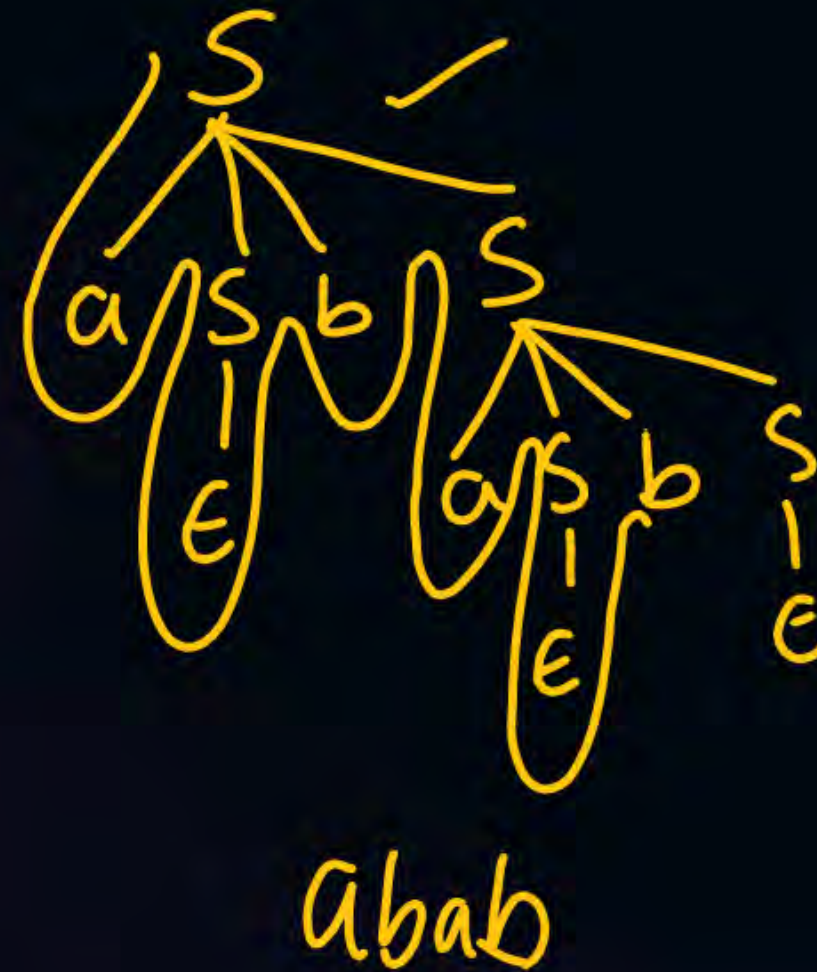
$$S \rightarrow aS / Sa / a$$

aa



$S \rightarrow aSbS / bSaS / \epsilon \rightarrow \text{amb}$

$w = \underline{abab}$ ✓



✓
 $R \rightarrow R+R / R.R / R^* / a / b / \epsilon \Rightarrow$ grammar for Regular expression

It generates any Reg exp.

$a + a.a$

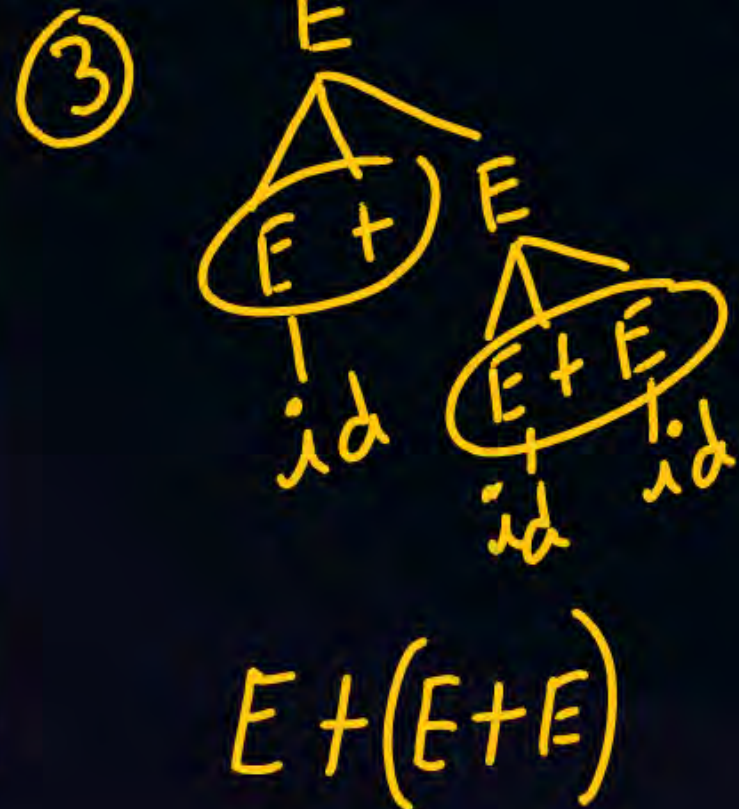
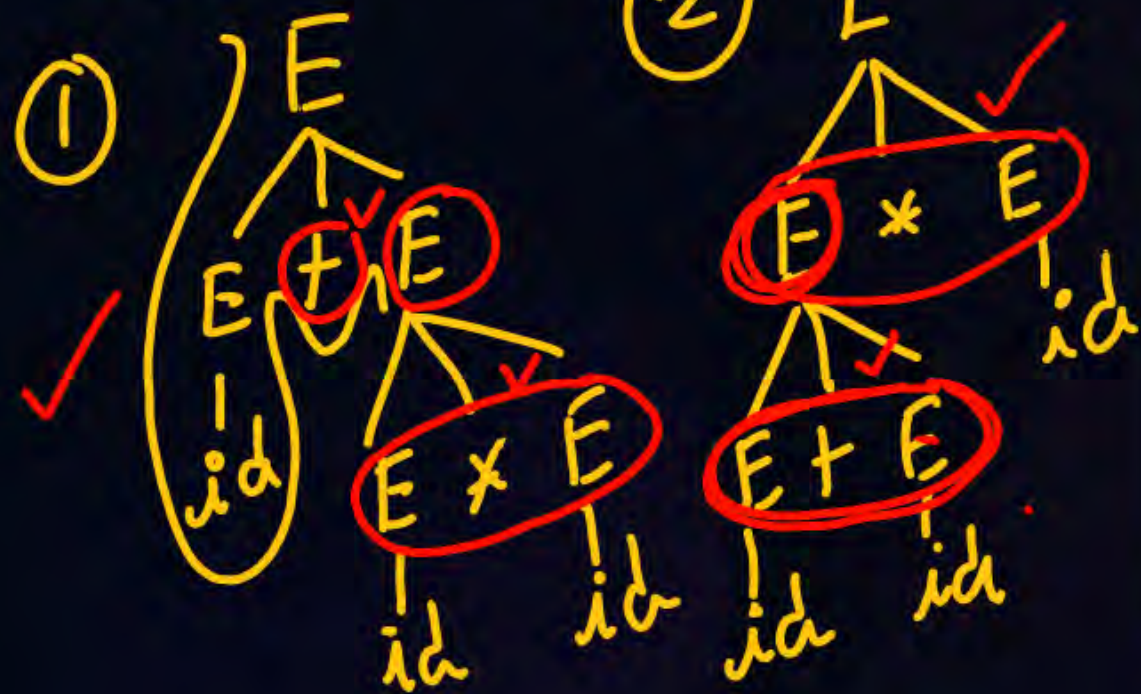


Some of the ambiguous grammars can be converted to unambiguous grammars by using disambiguating rules. 

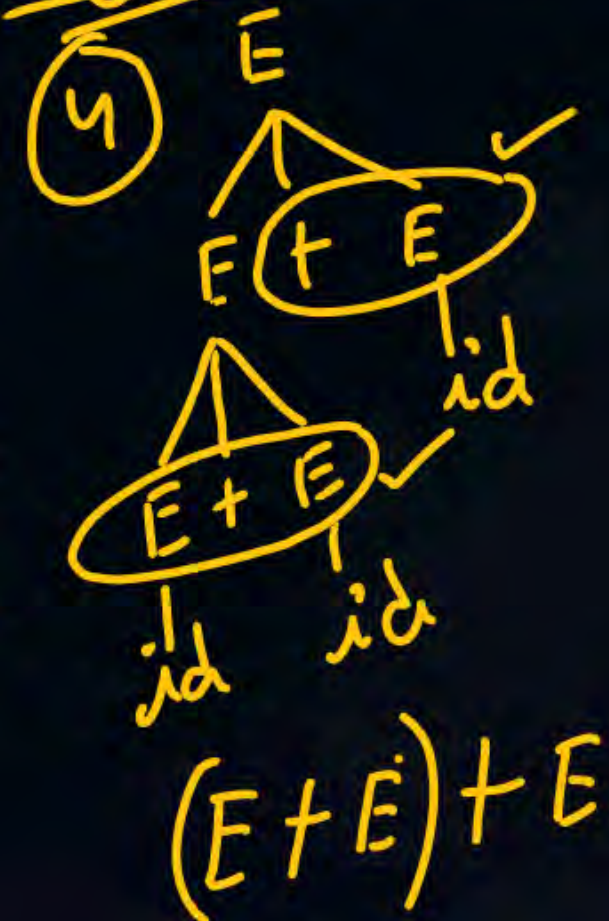
$E \rightarrow E + E$
 $\quad \quad \quad / E * E$
 $\quad \quad \quad / id$

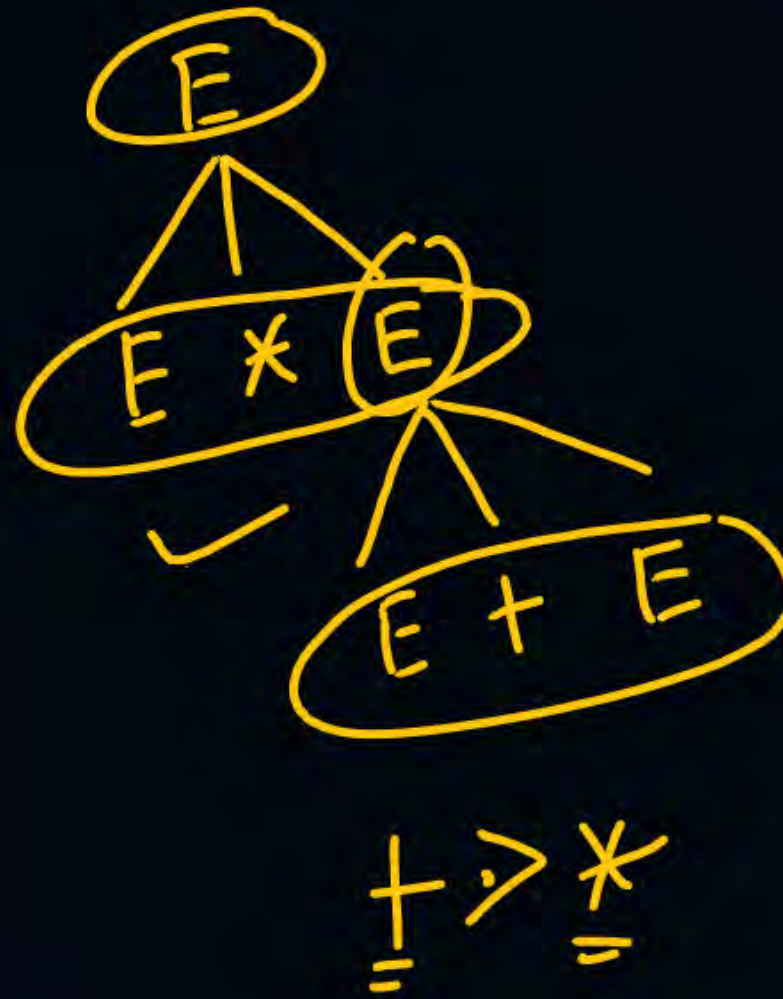
$$2 + 3 \times 4 = 14$$

$id + id * id$ (Precedence fail)



$id + id + id$ (associativity fail) $+ > +$



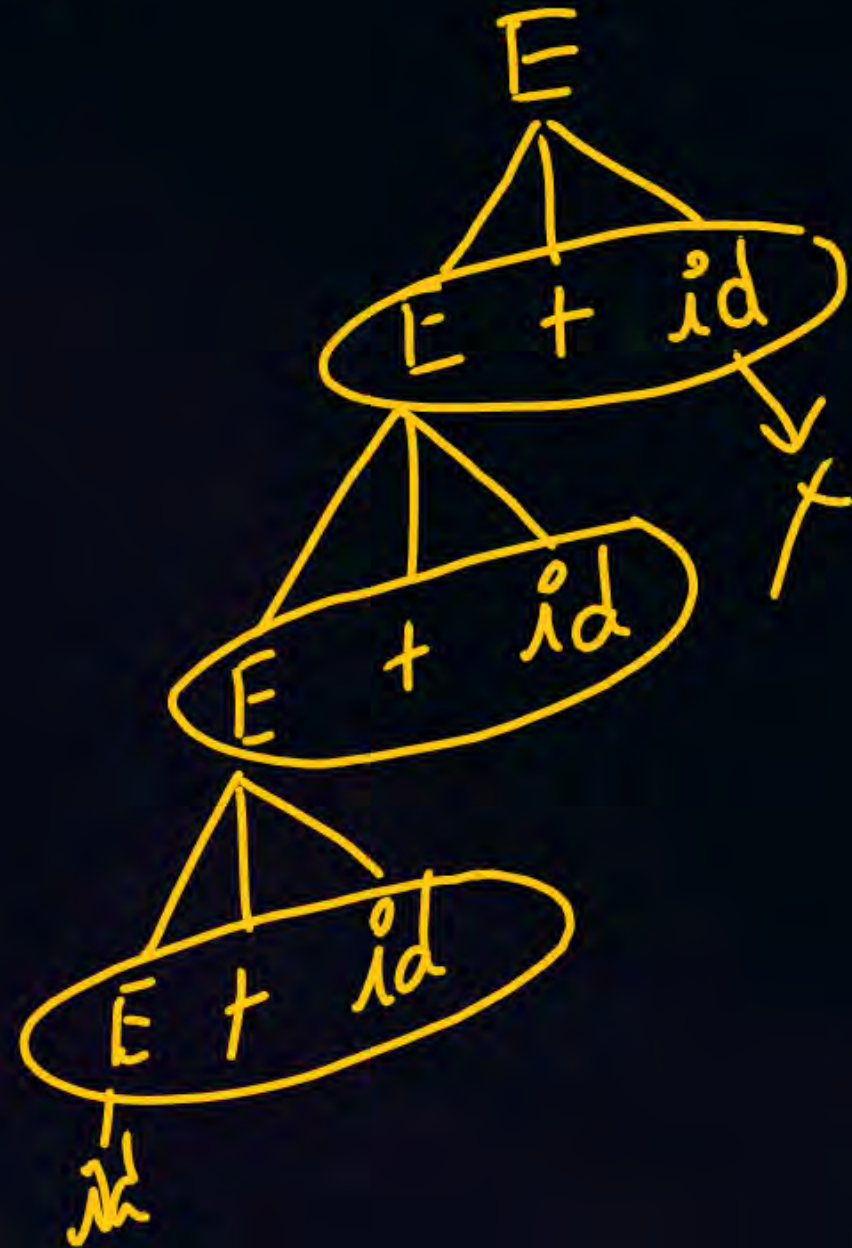


$$id + id + id$$

$$E \rightarrow \check{E} + (\check{E}) / id$$



$$E \rightarrow \check{E} + id / id$$



Left recursive ✓
 → left associative ✓

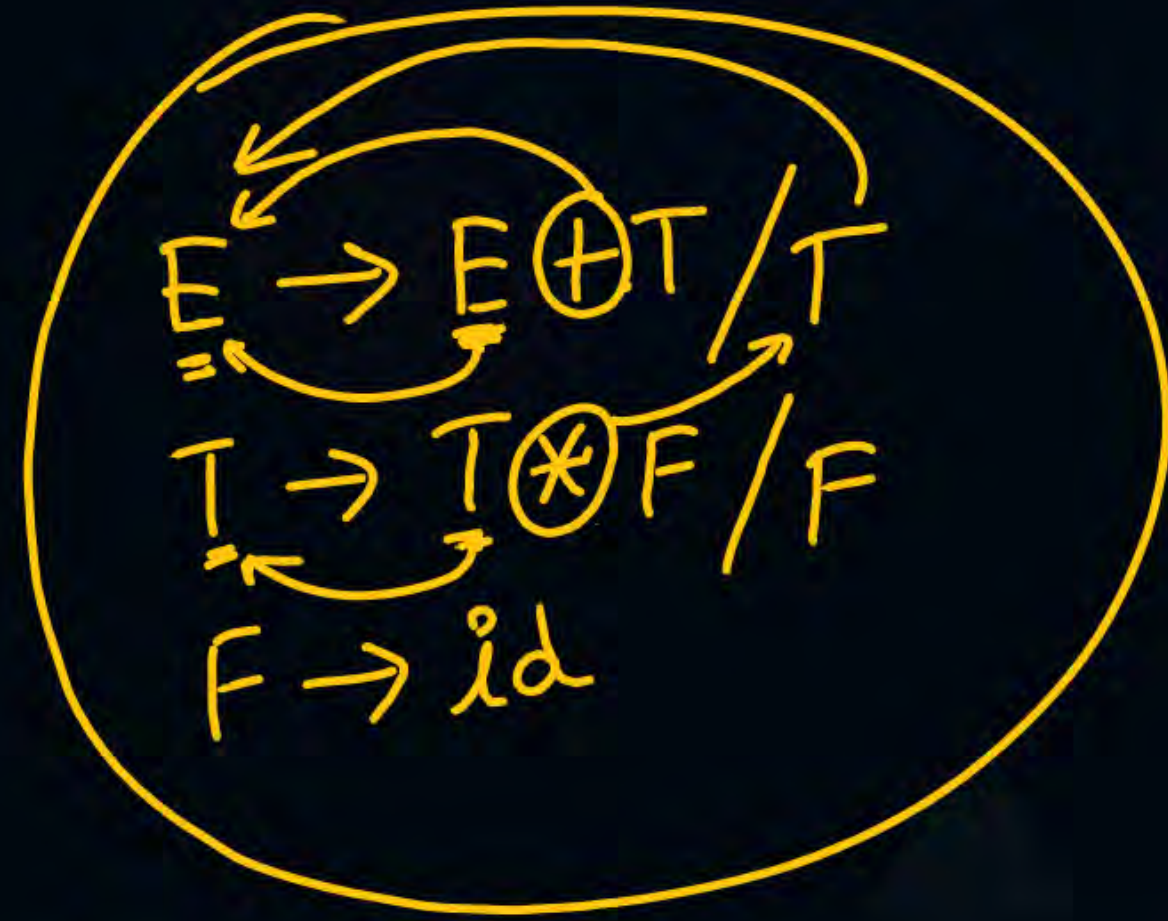
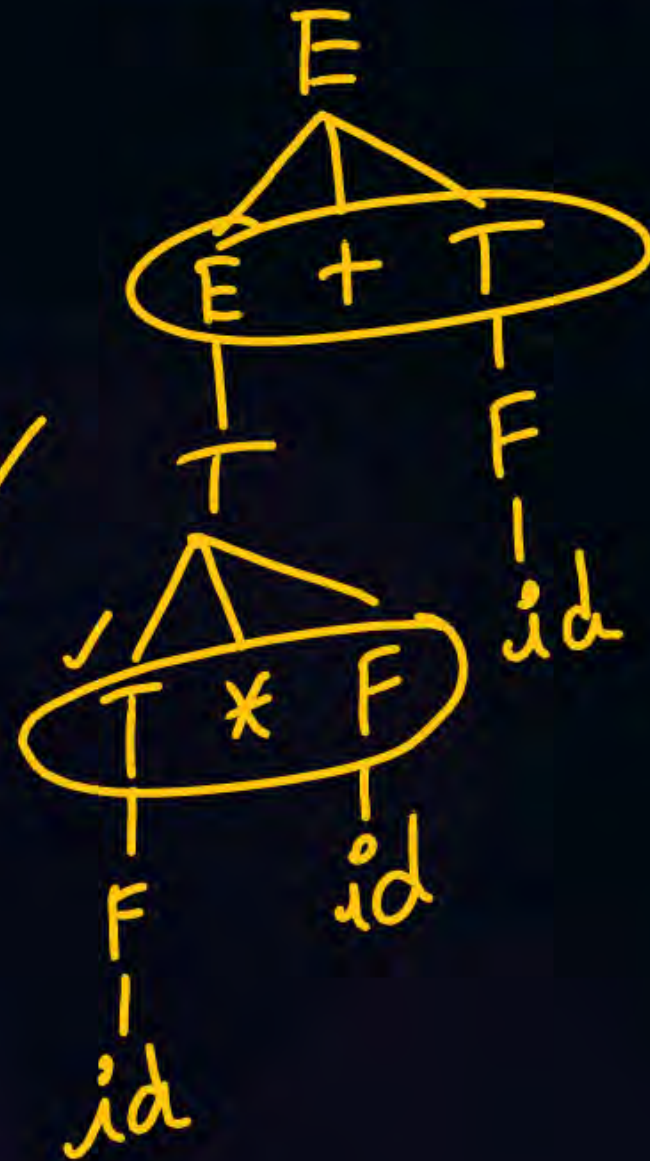
$id * id + id$



id + id * id



id * id + id



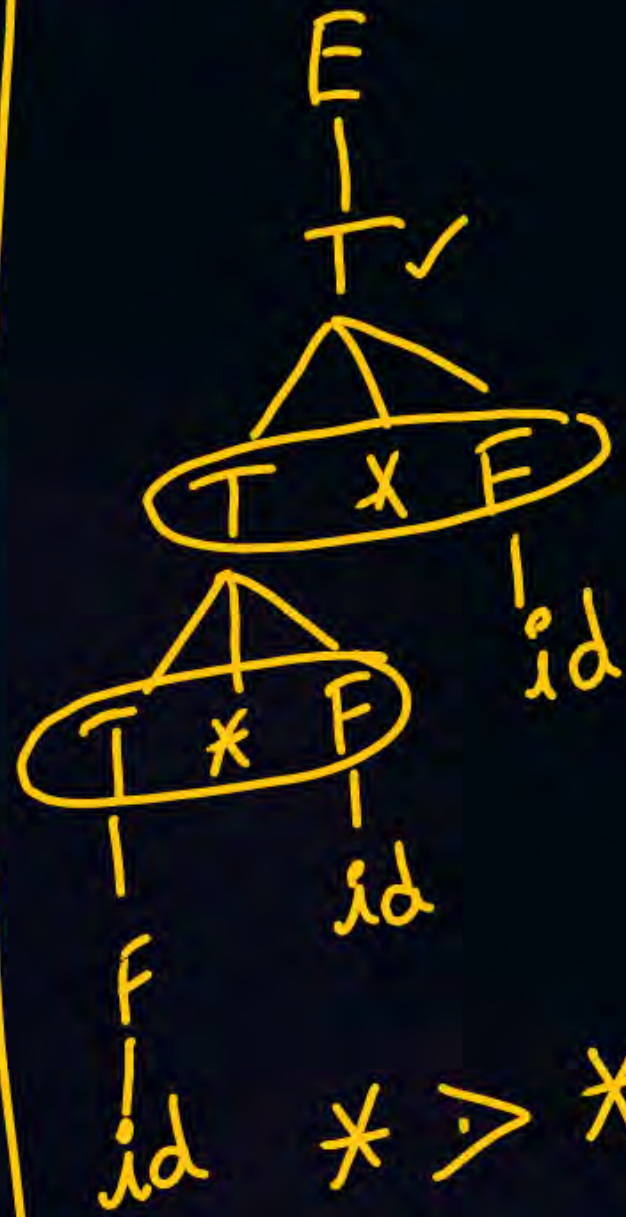
$* > +$

$id + id + id$

$id * id * id$



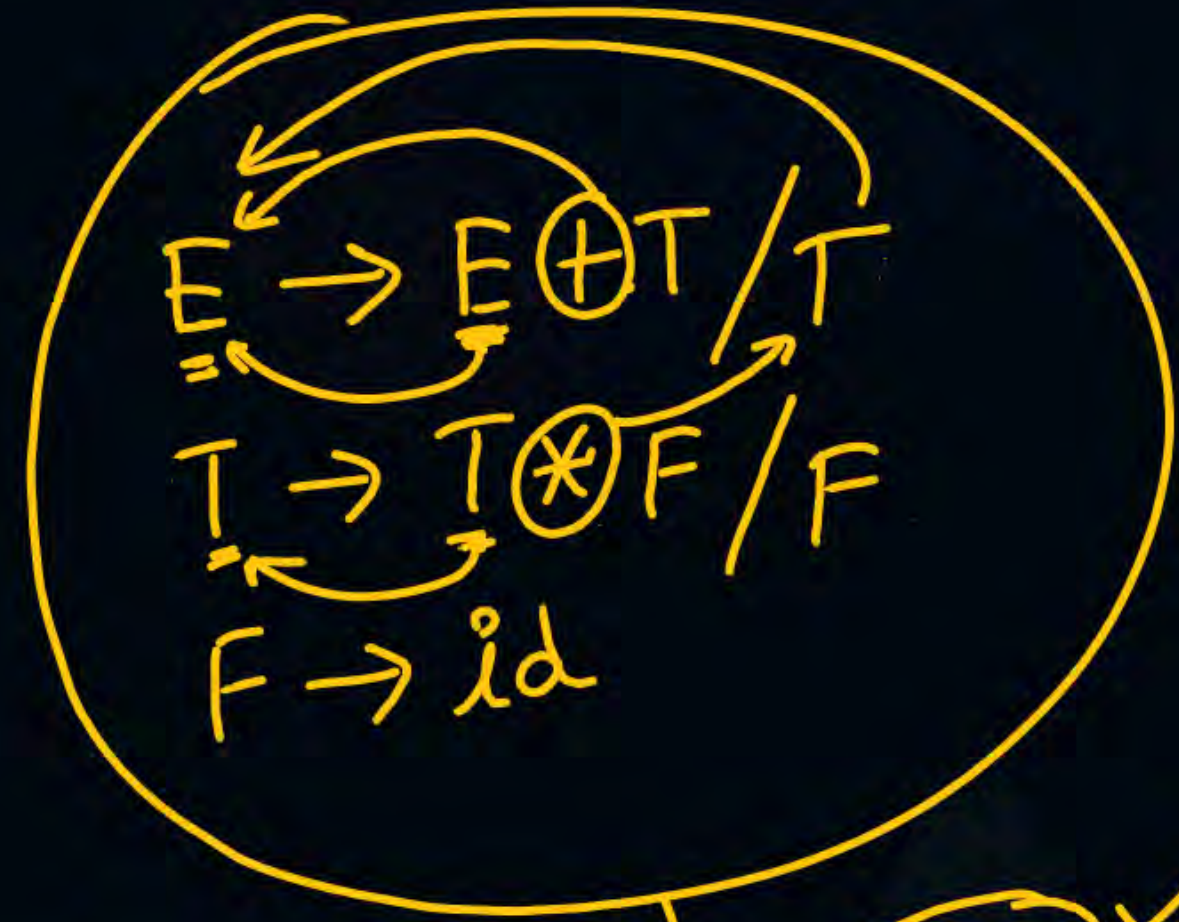
$+ > +$



$* > *$

$* > +$

↑



unambi ✓

$2^{\uparrow 3} = 2^3$
 $(2^{\uparrow (3^{\uparrow 2})})$
 $(2^{\uparrow 3})^2$

$$\begin{aligned}
 E &\rightarrow E + T / T \\
 T &\rightarrow T * F / F \\
 F &\rightarrow G \mid F / G \\
 G &\rightarrow id
 \end{aligned}$$

$\uparrow \triangleright * \triangleright +$
 $+ \triangleright +$
 $* \triangleright *$
 $\uparrow < \uparrow$

Boolean Expression:-

$bExpr \rightarrow bExpr \ \& \ bExpr$
 $\quad \quad \quad / \ bExpr \ \text{and} \ bExpr$
 $\quad \quad \quad / \ \text{not} \ bExpr$
 $\quad \quad \quad / \ \text{True}$
 $\quad \quad \quad / \ \text{false}$

$\underline{or} < \underline{and} < \underline{not}$

$E \rightarrow \underline{E} \ \& \ F / F$
 $F \rightarrow \underline{F} \ \underline{and} \ G / G$
 $G \rightarrow \underline{Not} \ G / \text{True} / \text{False}$

$\& > or$
 $\underline{and} > and$

$\downarrow$ unambiguous



THANK - YOU